

ACTIVIDAD PRÁCTICA — CLASE 8 · UNIDAD 5

Cazadores de Patrones en MiniRed

De la base de datos real a la regla de negocio, pasando por Apriori

Modalidad y entrega

Aspecto	Detalle
Formato de trabajo	Grupos de 3 a 4 integrantes (mismos grupos del TP1/TP2 si ya están conformados).
Herramientas de minería	Weka (obligatoria para al menos una corrida) + opcionalmente Python (mlxtend) o R (arules).
Herramienta de IA para presentar	Libre elección: ChatGPT, Claude, Gemini, Copilot u otra — se declara cuál se usó y para qué.
Formato de la presentación	Libre elección: landing interactivo, mini-dashboard, deck, notebook narrado — mientras cumpla los requisitos mínimos.
Entrega	Link o archivo de la presentación + informe corto (máx. 6 páginas) con las capturas de Weka/Python y los cálculos a mano.
Exposición	5 minutos por grupo, próxima clase, mostrando una regla curada y cómo se llegó a ella.

Fase 0 PREPARACIÓN DE DATOS

Objetivo: pasar de la tabla de hechos al formato que un algoritmo de asociación necesita.

Van a trabajar sobre MiniRed_DW, la misma base que ya tienen instalada y funcionando desde el TP de arquitectura. El punto de partida es Fact_Ventas con el grano en línea de ticket — exactamente la "base de transacciones" de la que hablamos en la Clase 8.

1. Recorten el universo. No se pide minar los 4.200 artículos de las 14 sucursales a la vez: elijan una sucursal, o un rango de fechas (por ejemplo, un trimestre), o un rubro. Justifiquen la elección en dos renglones — este es el mismo tipo de decisión de alcance que van a tener que defender en cualquier proyecto real.
2. Armen la consulta SQL que agrupe IdVenta (el ticket) contra los productos que contiene. Piensen el grano: la salida de esta consulta es una fila por ticket con la lista de productos, no una fila por línea de detalle.
3. Transformen esa salida al formato de canasta: una fila por transacción, una columna por ítem posible, con valores presente/ausente (o 1/0). Es el mismo paso que se explicó para Weka en la clase: NumericToNominal o Discretize si hace falta, y sacar cualquier columna que sea un identificador único, como el propio IdVenta.
4. Documenten cuántas transacciones y cuántos ítems distintos quedaron en su muestra final. Este número lo van a necesitar para justificar, más adelante, por qué eligieron el soporte mínimo que eligieron.

Atención: si trabajan a nivel de artículo individual y su muestra da muy dispersa (casi ninguna regla supera el soporte), no es un error — es exactamente el problema que se explicó en "Reglas multinivel". Suban un nivel en la jerarquía (a categoría, por ejemplo) antes de asumir que no hay patrones.

Fase 1 APLICAR EL ALGORITMO

Objetivo: correr Apriori sobre datos reales y probar, con números propios, que entienden lo que la herramienta está haciendo por dentro.

5. Carguen su matriz transaccional en Weka (Explorer → Associate → Apriori) y corran el algoritmo con un soporte mínimo inicial a su elección.
6. Repitan la corrida bajando el soporte mínimo al menos una vez más. Registren en una tabla cómo cambió la cantidad de reglas obtenidas entre ambas corridas — esto es evidencia directa de por qué "qué hacer con 40.000 reglas" no es una exageración.
7. De las reglas obtenidas en su mejor corrida, elijan 3 y calcúlenlas a mano: soporte, confianza y lift, con la misma fórmula y el mismo procedimiento que hicimos en clase con los cinco tickets. Comparen su resultado a mano contra lo que muestra Weka. Si no coincide, encuentren el error antes de seguir — no se avanza con un número que no se puede reproducir.
8. Como plus (no obligatorio, suma en la dimensión de presentación): corran la misma matriz con FP-Growth, en Weka o en Python con mlxtend, y comparen tiempos de ejecución y si las reglas resultantes coinciden con las de Apriori.

Los tres errores de siempre, tal como se vieron en clase: un atributo numérico sin discretizar deja Apriori deshabilitado sin explicación; dejar el identificador de ticket como atributo hace que

el algoritmo no encuentre nada o se cuelgue; y el soporte por defecto de Weka suele ser demasiado alto para encontrar algo interesante en una primera corrida. Si les pasa alguno de los tres, ya saben dónde mirar.

Fase 2 CURAR LAS REGLAS

Objetivo: aplicar el criterio, no solo el algoritmo.

De todas las reglas que les devolvió Weka, apliquen los seis pasos que vimos en "Qué hacer con 40.000 reglas" y quédense con las 5 a 8 que realmente le llevarían a la gerencia de MiniRed. Para cada una de las reglas descartadas en el camino, un renglón alcanza para explicar por qué se descartó — no hace falta un párrafo por regla.

Paso	Qué hacer con sus propias reglas
1. Descartar lo trivial	¿Hay alguna regla tipo "pan $\rightarrow$ manteca" que cualquiera del negocio ya sabía?
2. Descartar lo redundante	¿Hay una regla $\{A,B\} \rightarrow \{C\}$ que no aporta nada respecto de $\{A\} \rightarrow \{C\}$?
3. Ordenar por lift	Reordenen su lista completa por lift descendente, no por confianza.
4. Mecanismo plausible	Para las que sobrevivan, ¿hay una explicación de negocio posible?
5. Accionabilidad	¿Qué haría MiniRed distinto si esta regla es cierta?
6. Validar en otro período	Si el alcance lo permite, corran la misma regla sobre otro rango de fechas y vean si se sostiene.

Fase 3 PRESENTAR CON AYUDA DE UNA IA

Objetivo: comunicar el hallazgo, no solo tenerlo.

Acá la elección es de ustedes, igual que en la actividad de la Clase 4: eligen la IA que van a usar como asistente y el formato en el que van a mostrar sus reglas curadas. Puede ser un landing interactivo donde se navegan las combinaciones, un mini-dashboard, una presentación narrada, un notebook comentado — lo que mejor les sirva para explicarlo. Lo que no es libre son los contenidos mínimos que tiene que incluir, sea cual sea el formato:

- **La pregunta de negocio:** qué sucursal, período o recorte de MiniRed eligieron trabajar, y por qué.
- **Los datos de entrada:** una captura o extracto de cómo quedó la matriz transaccional que armaron.
- **El proceso de minería:** capturas de Weka (o de su script de Python) mostrando al menos dos corridas con distinto soporte.
- **La verificación manual:** el cálculo a mano de las 3 reglas de la Fase 1, mostrando que coincide con la herramienta.
- **Las reglas curadas:** las 5 a 8 reglas finales con soporte, confianza y lift, y una frase de recomendación de negocio por cada una.

- **El rol de la IA:** qué generó o ayudó a construir la IA elegida (texto, código, diseño visual) y qué tuvieron que corregir o verificar ustedes.

Sobre el uso de la IA: usarla para redactar, generar código de la matriz transaccional, o diseñar la interfaz está permitido y esperado. Lo que no se acepta es pedirle a la IA que invente las reglas o los números: soporte, confianza y lift tienen que salir de Weka o Python corriendo sobre los datos reales de MiniRed_DW, nunca de lo que una IA generativa "cree" que sería un buen ejemplo.

Fase 4 CONEXIÓN CON RA4

Objetivo: una reflexión escrita corta (media página), individual, que cada integrante entrega por separado dentro del informe grupal.

9. ¿Qué proporción del tiempo total de la actividad se fue en la Fase 0 (preparación de datos) comparado con correr el algoritmo en sí? Conecten esto con lo que dice la teoría de la Unidad 4 sobre por qué la preparación de datos suele consumir la mayor parte del esfuerzo en un proyecto de minería.
10. Piensen en la sección de advertencias de la Clase 8. De las seis formas de sacar la conclusión equivocada, ¿cuál sintieron que estuvieron más cerca de cometer con sus propios datos, y qué hicieron para evitarlo?
11. Un párrafo sobre el rol de la IA generativa en este trabajo: ¿en qué punto confiar ciegamente en lo que generó hubiera sido un riesgo, y cómo lo verificaron?

Qué no se acepta

- presentar reglas con buena confianza sin mostrar también el lift.
- usar la IA para inventar valores de soporte, confianza o lift sin correrlos sobre datos reales.
- una matriz transaccional que incluya el identificador de ticket como atributo (revisar la Clase 8, sección Weka).
- copiar el ejemplo de los cinco tickets de la clase en lugar de trabajar con datos de MiniRed_DW.
- un informe sin ninguna captura de Weka o del script usado — el proceso tiene que poder verificarse.